

An Entropy-Based Privacy-Preserving Federated Deep Reinforcement Learning Framework for Task Offloading in Vehicular Edge Computing Networks

Yishan Chen^{ID}, Jianwei Zhang^{ID}, Wenshuo Dai, Junxiao Han^{ID}, *Member, IEEE*,
Miaojiang Chen^{ID}, *Member, IEEE*, Zhiquan Liu^{ID}, *Senior Member, IEEE*, and Ahmed Farouk^{ID}

Abstract—With the rapid evolution of 5G and the ongoing development of 6G technologies, the Internet of Vehicles (IoV) is expected to play a critical role in next-generation intelligent transportation systems. Applications, such as autonomous driving, augmented reality, and smart mobility not only require ultralow latency and high computational efficiency, but also demand enhanced trustworthiness and privacy assurance. To address these demands, vehicular edge computing (VEC) has emerged as a foundational paradigm for 6G-IoT, enabling intelligent services by offloading tasks from vehicles to edge nodes. However, task offloading in IoV-VEC systems still faces critical challenges, including the need for responsible AI decision-making under dynamic network conditions and the protection of sensitive vehicular data. This article proposes federated vehicle task offloading (FedVTO), a privacy-preserving FedVTO framework that integrates federated learning (FL) and deep reinforcement learning (DRL) to optimize task offloading decisions and resource allocation strategies in VEC networks. By incorporating information entropy models and dynamically adjusting weighting parameters using an entropy-based method within a three-tier architecture (vehicles, roadside unit, and cloud server), FedVTO minimizes latency, energy consumption, and privacy leakage. Experimental results show that FedVTO significantly improves task offloading efficiency and mitigates privacy risks compared to traditional methods in dynamic VEC environments.

Index Terms—Deep reinforcement learning (DRL), federated learning (FL), privacy preservation, task offloading, vehicular edge computing (VEC).

Received 22 June 2025; revised 8 July 2025 and 13 August 2025; accepted 22 August 2025. Date of publication 26 August 2025; date of current version 23 February 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62302200 and Grant 62462002; in part by the Natural Science Foundation of Jiangxi Province under Grant 20232BAB212014; in part by the Jiangxi Provincial Key Laboratory of Multidimensional Intelligent Perception and Control under Grant 2024SSY03161; and in part by the National Science Foundation for Distinguished Young Scholars of Jiangxi Province under Grant 20252BAC220009. (*Corresponding author: Yishan Chen.*)

Yishan Chen, Jianwei Zhang, and Wenshuo Dai are with the School of Information Engineering, Jiangxi University of Science and Technology, Ganzhou 341000, China (e-mail: chenys@jxust.edu.cn; Zhangjw@mail.jxust.edu.cn; daiws@mail.jxust.edu.cn).

Junxiao Han is with the School of Computer and Computing Science, Hangzhou City University, Hangzhou 310015, China (e-mail: hanjx@hczu.edu.cn).

Miaojiang Chen is with the School of Computer, Electronics and Information, Guangxi University, Nanning, China, and also with the Guangxi Key Laboratory of Multimedia Communications and Network Technology, Nanning 530004, China (e-mail: mjchen_cs@gxu.edu.cn).

Zhiquan Liu is with the College of Cyber Security, Jinan University, Guangzhou 510632, China (e-mail: zqliu@vip.qq.com).

Ahmed Farouk is with the Faculty of Computers and Artificial Intelligence, Hurgada University, Hurgada 1974531, Egypt (e-mail: ahmed.farouk@sci.svu.edu.eg).

Digital Object Identifier 10.1109/IIOT.2025.3602798

I. INTRODUCTION

RECENT advancements in 5G and the rise of 6G technologies have accelerated the growth of the Internet of Vehicles (IoV), enabling compute-intensive, latency-critical applications, such as autonomous driving [1], augmented reality [2], and intelligent transportation systems [3]. These applications demand ultralow latency, high reliability, strong trustworthiness, and strict data privacy in 6G-IoT ecosystems. Traditional cloud architectures struggle to meet these requirements due to bandwidth bottlenecks, energy inefficiency, and privacy risks [4], highlighting the need for responsible and trustworthy edge-centric solutions.

To address these challenges, vehicular edge computing (VEC) has emerged as a key paradigm for 6G-IoT, providing computation and storage resources closer to end devices to support latency-sensitive intelligent services [5], [6], [7]. By deploying VEC nodes at the network edge, vehicles can offload compute-intensive tasks locally, thereby reducing transmission latency and energy consumption while mitigating the burden on centralized cloud infrastructures [8]. However, the dynamic and heterogeneous nature of vehicular networks—characterized by fluctuating traffic patterns, variable channel conditions, and node mobility—poses significant challenges to designing reliable and trustworthy offloading strategies. In addition, the diverse hardware capabilities and stringent QoS requirements in semi-autonomous driving applications [9] complicate system design, making it essential to employ adaptive AI-driven mechanisms for real-time resource management, data routing, and computation scheduling. The growing demand for data-intensive services, including high-resolution video transmission and real-time sensor fusion, places mounting pressure on edge resources, necessitating rapid and intelligent adaptation under varying load conditions [10]. These challenges underscore the need for robust optimization approaches capable of adapting dynamically to temporal variations in network states. Furthermore, system design must balance the limited onboard resources of vehicles with the capabilities of VEC infrastructure, particularly given the high mobility of vehicles and their short dwell time within any single edge node's coverage area.

At the same time, ensuring privacy and trust has become a pressing concern in IoV-enabled VEC systems, as offloaded tasks often involve sensitive information, such as vehicular trajectories, user behavior, and multimedia content [11].

Federated learning (FL) has proven effective in protecting data privacy by training models locally on distributed devices and transmitting only model updates, thereby avoiding direct transmission of raw user data [12], [13]. Nevertheless, applying FL in vehicular networks introduces challenges, such as non-IID data distributions, intermittent vehicle-to-infrastructure connectivity, and increased communication overhead from frequent model updates. Meanwhile, deep reinforcement learning (DRL) techniques—particularly those tailored to continuous action spaces—have shown significant potential in managing complex interactions among vehicles, edge nodes, and cloud infrastructures [14], [15]. Integrating FL and DRL offers a promising path toward adaptive, privacy-aware offloading strategies that jointly optimize decisions across vehicles and edge nodes. However, vehicle mobility requires careful coordination in scheduling and bandwidth allocation to maintain system reliability and trustworthiness. For practical deployment, privacy-preserving techniques—such as differential privacy (DP) and secure multiparty computation—must maintain model utility while minimizing communication overhead. These considerations emphasize the need for a trustworthy, integrated AI framework that effectively balances privacy, latency, energy efficiency, and resource constraints in 6G-IoT environments.

In this article, we propose federated vehicle task offloading (FedVTO), a privacy-aware and trustworthy FedVTO framework tailored for VEC in next-generation 6G-IoT networks. The framework integrates FL with DRL to support intelligent and adaptive offloading decisions within a hierarchical three-tier architecture comprising vehicles, roadside units (RSUs), and cloud servers. By integrating privacy-preserving information entropy models and context-aware resource allocation strategies, FedVTO aims to reduce latency and energy consumption while ensuring robust privacy guarantees. Experimental evaluations demonstrate that FedVTO significantly improves task offloading efficiency and effectively mitigates privacy risks under highly dynamic IoV conditions. The main contributions of this work are summarized as follows:

- 1) We establish a comprehensive three-tier VEC model that captures the interactions among vehicles, RSUs, and cloud servers. This model integrates privacy-preserving offloading strategies with information entropy metrics to effectively monitor and mitigate potential data leakage in dynamic vehicular environments.
- 2) A novel task offloading framework is proposed that integrates the Twin Delayed deep deterministic policy gradient (DDPG) (TD3) algorithm with FL and DP. This framework dynamically adjusts the weighting parameters using an entropy-based method, enabling adaptive resource allocation in response to changing network conditions and ensuring enhanced system performance.
- 3) Extensive experimental results are provided, validating the effectiveness of the FedVTO framework in reducing latency, energy consumption, and privacy leakage. A comparative analysis with baseline methods demonstrates that FedVTO outperforms existing solutions,

particularly in dynamic and unpredictable vehicular environments.

The remainder of this article is structured as follows. Section II provides a review of related literature in the areas of task offloading and privacy-preserving methods. Section III presents the system model and problem formulation in detail. Section IV introduces the proposed federated framework and DRL-based optimization method. Section V presents the experimental evaluation of our framework, and Section VI concludes this article.

II. RELATED WORK

This section reviews the relevant literature from two primary perspectives: 1) task offloading strategies in VEC and 2) privacy-preserving mechanisms integrated into offloading frameworks. By examining the strengths and limitations in both areas, we clarify the contributions and motivations of the proposed methods in this article.

A. Task Offloading in VEC

Task offloading in VEC has been a critical focus for improving system performance, early research primarily focused on optimizing specific objectives like energy consumption and latency. Long et al. [16] explored the tradeoff between energy usage and task delay in VEC, proposing an offloading mechanism that dynamically adjusts resource allocation based on vehicles' mobility. Similarly, Liu et al. [17] studied a device-edge-cloud collaborative system with subtask dependencies, formulating a joint resource allocation and task offloading scheme to improve system throughput. In satellite-based IoT and UAV-assisted MEC networks, researchers have demonstrated that integrated architectures significantly improve offloading efficiency and reduce operational costs [18], [19].

As decision-making frameworks become more complex, techniques such as DRL and FL have gained considerable traction. For example, federated DRL has been employed to optimize multimedia task offloading in vehicular networks and to manage partially offloaded tasks in dynamic network environments [15], [20], [21]. Furthermore, multiagent DRL has been successfully applied to coordinate offloading decisions in satellite-based MEC, allowing agents to share resources efficiently under constrained computing resources [22]. Game-theoretic models have also been employed to optimize task allocation in vehicular networks, addressing uncertainties in vehicle movement, resource availability, and task completion times [23]. More recent work has introduced partial task offloading and resource allocation schemes in dynamic VEC environments using heuristic algorithms, optimizing task offloading while managing vehicle dynamics [24], [25]. Other studies applied game theory to UAV-assisted MEC systems, optimizing task offloading and resource pricing to enhance energy efficiency and computational resource utilization [26]. Nevertheless, despite these performance gains, many studies either overlook or only superficially address privacy concerns, which are becoming increasingly critical as the volume and sensitivity of offloaded data continue to grow.

B. Privacy Protection in Task Offloading

In addition to optimizing task offloading, privacy protection has become a critical concern in VEC. Since offloaded tasks often involve sensitive user data, any breach of this data could lead to significant privacy violations. Privacy-aware offloading strategies have been developed to assess and mitigate the risks of data exposure [27], [28]. Zhang et al. [29] proposed a privacy-aware offloading approach that leverages privacy entropy to evaluate and mitigate potential data exposure in multiaccess edge environments. In addition, Liu et al. [30] studied secure offloading in multi-eavesdropper MEC networks, designing a satisfaction maximization mechanism that combines energy consumption, task latency, and eavesdropping threats. FL and DP have become key methods for safeguarding user privacy. By enabling local model training on devices rather than transmitting raw data to a central server, FL significantly reduces the risk of data breaches. Some studies have incorporated DP into FL to protect parameter exchanges during model training, preventing privacy leakage. These methods have been applied in industrial IoT environments, where task offloading schemes protect user privacy while maintaining high efficiency [31], [32]. In a similar vein, Zhao et al. [33] devised a personalized and DP-aware offloading solution for video streaming in MEC, introducing noise-perturbation strategies to safeguard personal usage patterns. Additionally, incentive mechanisms have been introduced to ensure that nodes truthfully share model updates while maintaining privacy by hiding identifiable information [34]. To protect users' location data during task offloading, several studies have employed perturbation-based and encryption-based techniques. These methods aim to conceal users' exact locations and prevent the leakage of sensitive information, ensuring privacy during task execution [28]. Further approaches combine cryptographic methods with FL to ensure data confidentiality while reducing communication overhead in MEC systems [35], [36], [37], [38].

Despite advances in VEC task offloading, most contemporary methods focus on optimizing single metrics—such as latency or energy—while giving privacy only secondary consideration. This induces suboptimal Pareto solutions where performance gains potentially compromise sensitive vehicular data. Although FL circumvents raw data transmission, it fails to mitigate inherent privacy risks in model updates, which may leak vehicle locations or task information. Moreover, existing frameworks lack theoretically grounded mechanisms to dynamically quantify and balance privacy against latency and energy under resource-constrained, time-varying network conditions. In contrast, our FedVTO framework unifies FL with DRL through an entropy-based privacy quantification model. By adaptively weighting these competing objectives, FedVTO enables trustworthy task offloading that simultaneously achieves performance optimization and rigorous privacy preservation.

III. SYSTEM MODEL AND PROBLEM FORMULATION

In this section, we present a three-tier VEC architecture tailored for privacy-preserving FedVTO framework, as illustrated

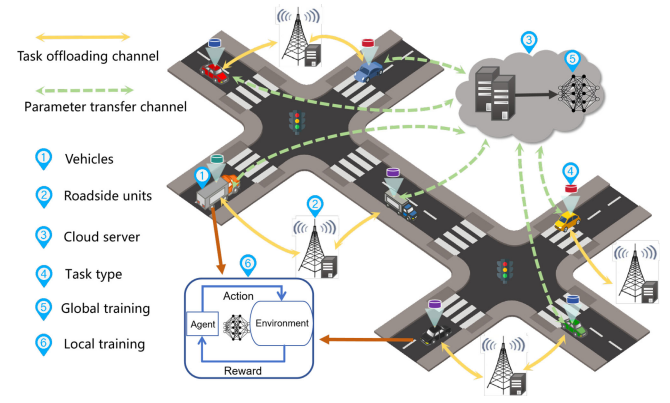


Fig. 1. Three-tier VEC architecture for FedVTO framework.

in Fig. 1. The proposed architecture includes Vehicles layer, RSUs layer, and Cloud server layer. Each layer plays a distinct role in enabling efficient and secure task offloading:

- 1) *Vehicles Layer*: The vehicle layer comprises a set of N vehicles, denoted as $\mathcal{V} = \{v_1, v_2, \dots, v_N\}$. Vehicles in this layer are highly mobile and have limited computing resources due to their size and power constraints. Each vehicle can generate computational tasks based on its real-time processing needs, which vary depending on factors, such as vehicle speed, location, and application requirements. At any time slot t , each vehicle v_i may generate tasks from different types, $T = \{t_1, t_2, \dots, t_K\}$, such as video processing, image recognition, or sensor data aggregation. Each task type has an associated task size, denoted as $B_s = \{B_1, B_2, \dots, B_k\}$, $k \in \{1, 2, \dots, K\}$, which influences the computational load and transmission requirements for offloading.
- 2) *RSUs Layer*: The RSU layer includes M RSUs, denoted as $RSUs = \{r_1, r_2, \dots, r_M\}$. The RSU layer is responsible for providing edge computing resources that offload computational tasks from vehicles. RSUs dynamically adjust to network conditions and vehicle mobility by continuously monitoring local task loads, channel conditions, and the availability of computation resources. When a vehicle enters the coverage area of an RSU, it can select the optimal RSU based on these factors, allowing the RSU to provide timely resources for task offloading.
- 3) *Cloud Server Layer*: The cloud server coordinates the global offloading strategy by collecting periodic summaries from RSUs. It processes these summaries to estimate future network states and makes global task offloading decisions accordingly. While RSUs handle immediate, latency-sensitive decisions, the cloud uses a prediction mechanism based on historical data to optimize task scheduling across vehicles and RSUs. The cloud server ensures that tasks are offloaded efficiently while maintaining the privacy of vehicle data through FL and DP techniques. To enhance clarity, a summary of the key notations used in the model is provided in Table I.

TABLE I
KEY SYSTEM PARAMETERS

Notation	Description
V	Set of vehicles
$RSUs$	Set of roadside units
C	Cloud server
$T_i(t)$	Task generated by vehicle v_i at time t
$B_i^{\text{up}}(t)$	Uplink bandwidth for v_i
$B_i^{\text{down}}(t)$	Downlink bandwidth for v_i
P_i^{up}	Uplink transmit power of v_i
P_j^{down}	Downlink transmit power of RSU r_j
$T_i^{\text{local}}(t)$	Local computation delay
$T_i^{\text{edge}}(t)$	Edge computation delay
$E_i^{\text{local}}(t)$	Local computation energy
$E_i^{\text{edge}}(t)$	Edge computation energy
$I_i(t)$	Interference at vehicles v_i
$I_j(t)$	Interference at RSUs r_j
$H(V_{\text{source}})$	Privacy entropy at vehicles
$H(R_{\text{sink}})$	Privacy entropy at RSUs
$\delta_{\text{type}}(t)$	Task type
$\delta_{\text{size}}^{\text{up}}(t)$	Upload data size
$\delta_{\text{size}}^{\text{res}}(t)$	Result data size
$\delta_d(t)$	Task deadline
$\xi_i(t)$	Offloading decision variable
f_i	CPU frequency of vehicle v_i
f_{r_j}	CPU frequency of RSU r_j
c_i^{local}	CPU cycles/bit for local computation
c_j^{edge}	CPU cycles/bit for edge computation
κ	Local energy factor
ε	Energy cost/bit at RSUs
$h_{i,j}(t)$	Channel gain between v_i and r_j

To ensure practical deployment in highly dynamic vehicular environments, the FedVTO framework employs a hierarchical and semi-decentralized coordination scheme. Instead of relying on continuous full-state feedback from vehicles or RSUs, the cloud server receives periodic state digests from RSUs. These digests are lightweight summaries that contain localized statistics, including average uplink/downlink channel quality, aggregated task load, vehicle density, and queuing conditions within the RSU's coverage area. This mechanism significantly reduces control signaling overhead and avoids the impracticality of full real-time synchronization. To further minimize communication burden, RSUs transmit only incremental updates when significant state changes occur, using lightweight compressed signaling protocols. Meanwhile, the cloud server does not assume perfectly fresh information. It employs short-term prediction mechanisms based on previously received digests to estimate the near-future network state. This compensates for possible digest transmission delays or partial staleness. In addition, time-sensitive decisions, including immediate task scheduling, RSU association, and channel selection, are delegated to the RSU layer. This ensures low-latency responsiveness without relying on cloud feedback. Only long-term strategies, such as global task distribution patterns and interzone resource planning, are computed at the cloud level.

A. Task Model

In the VEC framework, the computational task generated by the vehicle v_i at the moment of t is denoted as $T_i(t) = \{\delta_{\text{type}}(t), \delta_{\text{size}}^{\text{up}}(t), \delta_{\text{size}}^{\text{res}}(t), \delta_d(t)\}$, where $\delta_{\text{type}}(t)$ denotes the task type. Each value corresponds to a unique vehicle task type: $\delta_{\text{type}}(t) = 0$ denotes a text task, $\delta_{\text{type}}(t) = 1$ denotes a video task, $\delta_{\text{type}}(t) = 2$ denotes an audio task, and $\delta_{\text{type}}(t) = 3$ denotes image tasks. Each task type is associated with a specific data size, and the required computational resources vary depending on the task type and the hardware limitations of the vehicle.

The task parameters are as follows: $\delta_{\text{size}}^{\text{up}}(t)$ denotes the size of the uploaded data, $\delta_{\text{size}}^{\text{res}}(t)$ denotes the size of the computational result returned to the vehicle, and $\delta_d(t)$ denotes the latest time for the task to complete. We use the binary offloading decision variable $\xi_{j,i}(t) \in \{0, 1\}$ to determine the offloading strategy, where $j = 0$ represents local processing. If $\xi_{j,i}(t) = 1$, the task is offloaded to the RSU r_j for processing. Each task is indivisible and can only be assigned to one computing node, subject to the constraint: $\sum_{j=0}^M \xi_{j,i}(t) = 1 \forall i, t$.

B. Task Offloading Model

In the proposed VEC system, task offloading decisions are designed to optimize resource usage while safeguarding privacy. The model comprises two main components: 1) a computation model and 2) a communication model. When a task is offloaded to an RSU, efficient communication is critical for transmitting and retrieving task data.

1) *Computation Model*: Each vehicle can choose to process its tasks locally or offload them to nearby RSUs based on the task requirements and available resources.

Local Computation: When the binary offloading decision variable $\xi_i(t) = 0$, the vehicle v_i processes the task locally. The task completion time $T_i^{\text{local}}(t)$ is determined by the computational power of the vehicle, which depends on the local computation frequency f_i and the CPU cycles per bit required by the vehicle v_i , denoted by c_i^{local} . Therefore, the local computation delay of the task $T_i(t)$ can be expressed as

$$T_i^{\text{local}}(t) = \frac{\delta_{\text{size}}^{\text{up}}(t) \cdot c_i^{\text{local}}}{f_i}. \quad (1)$$

The energy consumed for local computation, $E_i^{\text{local}}(t)$, is calculated based on the power efficiency factor κ , where each CPU cycle consumes κf_i^2 energy. The total energy consumption for locally processing task $T_i(t)$ is given by

$$E_i^{\text{local}}(t) = \kappa f_i^2 \cdot \delta_{\text{size}}^{\text{up}}(t) \cdot c_i^{\text{local}}. \quad (2)$$

Offloading to RSU: If the vehicle v_i enters the coverage area of an RSU r_j and decides to offload the task (i.e., $\xi_i(t) = 1$), the task is transmitted to the RSU, where it waits for processing based on RSU availability. The computation delay $T_i^{\text{edge}}(t)$ for edge computation depends on the RSU's processing frequency f_{r_j} and the CPU cycles c_j^{edge} required

for the specific task type. The edge computation delay can be represented as

$$T_i^{\text{edge}}(t) = \frac{\delta_{\text{size}}^{\text{up}}(t) \cdot c_j^{\text{edge}}}{f_{r_j}} + T_i^{\text{wait}}(t) \quad (3)$$

where $T_i^{\text{wait}}(t)$ represents the queueing delay caused by tasks awaiting processing at the RSU. The energy consumed for edge computation, $E_i^{\text{edge}}(t)$, is determined by the processing energy cost per bit ε at the RSU, thus

$$E_i^{\text{edge}}(t) = \delta_{\text{size}}^{\text{up}}(t) \cdot \varepsilon. \quad (4)$$

2) *Communication Model*: This model outlines the transmission process for task offloading, encompassing both the uplink (from vehicle to RSU) and, when necessary, the downlink (from RSU to vehicle) phases. To improve flexibility and efficiency, we introduce dynamic bandwidth allocation variables $B_i^{\text{up}}(t)$ and $B_i^{\text{down}}(t)$ for each vehicle v_i at time t .

Uplink Transmission: To offload a task to the RSU, the vehicle transmits the task data $\delta_{\text{size}}^{\text{up}}(t)$ over the wireless uplink channel using allocated bandwidth $B_i^{\text{up}}(t)$. The uplink transmission rate between vehicle v_i and RSU r_j is given by

$$r_i^{\text{up}}(t) = B_i^{\text{up}}(t) \log_2 \left(1 + \frac{P_i^{\text{up}} h_{i,j}(t)}{\sigma^2 + I_i(t)} \right) \quad (5)$$

where $I_i(t)$ represents the interference experienced by vehicle v_i at time t , including co-channel interference and interference from other vehicles.

The uplink transmission delay $T_i^{\text{up}}(t)$ and energy consumption $E_i^{\text{up}}(t)$ are

$$T_i^{\text{up}}(t) = \frac{\delta_{\text{size}}^{\text{up}}(t)}{r_i^{\text{up}}(t)}, \quad E_i^{\text{up}}(t) = P_i^{\text{up}} \cdot T_i^{\text{up}}(t). \quad (6)$$

Downlink Transmission: After the task is processed by the RSU, the results of size $\delta_{\text{size}}^{\text{res}}(t)$ are returned to the vehicle using allocated bandwidth $B_i^{\text{down}}(t)$. The downlink transmission rate $r_i^{\text{down}}(t)$ between the RSU and vehicle is given by

$$r_i^{\text{down}}(t) = B_i^{\text{down}}(t) \log_2 \left(1 + \frac{P_j^{\text{down}} h_{j,i}(t)}{\sigma^2 + I_j(t)} \right) \quad (7)$$

where $I_j(t)$ represents the interference experienced by the RSU r_j during downlink transmission.

The downlink transmission delay $T_i^{\text{down}}(t)$ and energy consumption $E_i^{\text{down}}(t)$ are

$$T_i^{\text{down}}(t) = \frac{\delta_{\text{size}}^{\text{res}}(t)}{r_i^{\text{down}}(t)}, \quad E_i^{\text{down}}(t) = P_j^{\text{down}} \cdot T_i^{\text{down}}(t). \quad (8)$$

3) *Total Delay and Energy Consumption*: The total delay $T_i(t)$ and energy consumption $E_i(t)$ for task $T_i(t)$ are summarized as

$$T_i(t) = \begin{cases} T_i^{\text{local}}(t), & \xi_i(t) = 0 \\ T_i^{\text{up}}(t) + T_i^{\text{edge}}(t) + T_i^{\text{down}}(t), & \xi_i(t) = 1 \end{cases} \quad (9)$$

$$E_i(t) = \begin{cases} E_i^{\text{local}}(t), & \xi_i(t) = 0 \\ E_i^{\text{up}}(t) + E_i^{\text{edge}}(t) + E_i^{\text{down}}(t), & \xi_i(t) = 1 \end{cases} \quad (10)$$

We note that the current communication model focuses on instantaneous system parameters at each decision time slot. This formulation captures essential physical-layer characteristics under practical computational constraints. The impact of temporal channel variations and dynamic network behaviors is considered later in Section IV through sequential learning in our reinforcement learning framework.

C. Privacy-Preserving Information Entropy Model

Ensuring the privacy of task-related data during transmission and computation among vehicles, RSUs, and cloud servers is crucial. Such data often contains sensitive information, including vehicle locations and driving trajectories, which, if exposed, could lead to severe threats to both vehicular security and individual privacy. Therefore, privacy protection in vehicular network communications is essential to ensure that such sensitive data remains confidential. To address this challenge, we propose a privacy measurement method based on information entropy. Entropy, from information theory, quantifies the uncertainty or unpredictability of data. In the context of data transmission, higher entropy indicates that the data is more random and unpredictable, suggesting a higher risk of privacy leakage. Conversely, lower entropy means the data follows more predictable patterns, which could lead to privacy breaches if the data is exposed [29]. In our system, entropy is calculated at two key points: 1) the data generated by the vehicle (privacy source) and 2) the data received by the RSU (privacy sink). The difference in entropy between these two stages quantifies the potential for privacy leakage.

1) *Privacy Source*: For each task $T_i(t)$ generated by vehicle v_i , the corresponding data stream constitutes the privacy source $V_{\text{source}} = \{T_1(t), T_2(t), \dots, T_N(t)\}$. The entropy of this privacy source $H(V_{\text{source}})$ is defined as

$$H(V_{\text{source}}) = - \sum_{i=1}^N p(T_i) \log_2 p(T_i) \quad (11)$$

where $p(T_i)$ is the probability of task data $T_i(t)$ being collected, satisfying $0 \leq p(T_i) \leq 1$ and $\sum_{i=1}^N p(T_i) = 1$. This entropy $H(V_{\text{source}})$ reflects the level of privacy by representing the uncertainty in the task data generated by the vehicle.

2) *Privacy Sink*: During task offloading, the task data is transmitted to the RSUs, which act as the privacy sink. The probability distribution of task data received by the privacy sink R_{sink} is denoted as $q(R_j)$, where $j = 1, 2, \dots, M$ represents distinct RSUs. The entropy of the privacy sink $H(R_{\text{sink}})$ is defined as follows:

$$H(R_{\text{sink}}) = - \sum_{j=1}^M q(R_j) \log_2 q(R_j). \quad (12)$$

The value $H(R_{\text{sink}})$ reflects the uncertainty inherent in the task data received by the RSU.

3) *Privacy Leakage Information Entropy*: Privacy leakage can occur during the transmission of task data. To measure the extent of privacy leakage, we define privacy leakage information entropy $I(V_{\text{source}}; R_{\text{sink}})$, quantifying the entropy difference from the privacy source to the privacy sink

$$I(V_{\text{source}}; R_{\text{sink}}) = H(V_{\text{source}}) - H(V_{\text{source}} | R_{\text{sink}}) \quad (13)$$

where conditional entropy $H(V_{\text{source}} | R_{\text{sink}})$ represents the uncertainty in the privacy source given that the RSUs have received the data. It is defined as

$$H(V_{\text{source}} | R_{\text{sink}}) = - \sum_{i=1}^N \sum_{j=1}^M p(T_i, R_j) \log_2 p(T_i | R_j). \quad (14)$$

Here, $p(T_i, R_j)$ denotes the joint probability that vehicle task T_i is successfully transmitted to RSU R_j , and $p(T_i | R_j)$ represents the conditional probability of task T_i transmission given it is received by R_j . By evaluating the privacy information entropy $I(V_{\text{source}}; R_{\text{sink}})$, the degree of privacy preservation during task offloading can be quantitatively assessed, offering a theoretical foundation for designing effective privacy protection strategies. A larger value of $I(V_{\text{source}}; R_{\text{sink}})$ implies stronger privacy protection, as it reflects higher uncertainty and lower predictability of the transmitted data, thereby decreasing the likelihood of sensitive information being disclosed. Conversely, a smaller entropy value indicates greater data predictability and lower randomness, which corresponds to an increased risk of privacy leakage due to the higher chance of sensitive content being inferred. By integrating mutual information analysis into system design, we provide a principled framework for evaluating and enforcing privacy guarantees under dynamic communication and computation conditions.

D. Problem Formulation

In the previous sections, task offloading was modeled at the individual-task level, with separate metrics for delay, energy consumption, and privacy entropy. To optimize the overall system performance, it is necessary to aggregate these metrics across all tasks and vehicles. Therefore, the objective function combines the total delay, total energy consumption, and total privacy of the entire system.

The total system delay T_{total} is calculated by summing the delays of all tasks, whether processed locally or offloaded, across all vehicles and RSUs

$$T_{\text{total}} = \sum_{i=1}^N \sum_{t=1}^T T_i(t) \quad (15)$$

where $T_i(t)$ represents the delay for task $T_i(t)$ generated by vehicle v_i at time t .

The total energy consumption E_{total} is calculated by summing the energy consumed by all tasks across all vehicles and RSUs

$$E_{\text{total}} = \sum_{i=1}^N \sum_{t=1}^T E_i(t) \quad (16)$$

where $E_i(t)$ is the energy consumption for task $T_i(t)$, depending on whether it is processed locally or offloaded.

The total privacy entropy H_{privacy} is determined by aggregating the privacy entropy related to task offloading from the vehicles to RSUs

$$I_{\text{total}} = \sum_{i=1}^N I(V_{\text{source}}, R_{\text{sink}}) \quad (17)$$

where $I(V_{\text{source}}, R_{\text{sink}})$ measures the privacy leakage entropy, as defined in the privacy-preserving information entropy model.

Our objective is to minimize the overall system latency and energy consumption while maximizing privacy preservation. This involves decisions on offloading, communication resource allocation, and computational resource allocation. Specifically, the optimization problem can be formulated as

$$\min(T_{\text{total}} + E_{\text{total}}), \max I_{\text{total}} \quad (18)$$

$$\text{s.t. } T_i(t) \leq \delta_d(t) \quad \forall i, t \quad (18a)$$

$$\xi_i(t) \in \{0, 1\} \quad \forall i, t \quad (18b)$$

$$\sum_{j=0}^M \xi_{j,i}(t) = 1 \quad \forall i, t \quad (18c)$$

$$C_{v_i}(t) \leq C_{\text{max}}^{\text{local}}, C_{r_j}(t) \leq C_{\text{max}}^{\text{edge}} \quad \forall i, j, t \quad (18d)$$

$$\sum_{i=1}^{N(t)} B_i^{\text{up}}(t) \leq B_u, \sum_{i=1}^{N(t)} B_i^{\text{down}}(t) \leq B_d \quad \forall t. \quad (18e)$$

The deadline constraint for each task is represented by (18a). Each task must be completed within its specified deadline $\delta_d(t)$. The offloading decision variable is binary, as indicated by (18b), meaning each task can either be processed locally or offloaded. Equation (18c) ensures that each task is assigned to only one processing unit. The computing and communication resources of each vehicle and RSU are limited, as enforced by (18d). Equation (18e) guarantees the bandwidth resource constraints.

IV. APPROACH

This section outlines the optimization of task offloading in the FedVTO framework using the Twin-Delayed DDPG (TD3) algorithm and federated learning with differential privacy (FL-DP). DRL is used for making dynamic offloading decisions, while FL ensures privacy by allowing local model training, with DP further protecting user data during model aggregation. The following subsections detail the DRL-based optimization and the FL-based training framework.

A. DRL-based Task Offloading Optimization

To effectively solve this multiobjective optimization problem, we integrate task offloading decisions, communication resource allocation, and computational resource allocation into a unified framework, using DRL methods to address the high-dimensional state space. Specifically, the optimization objectives include minimizing latency and energy consumption while maximizing privacy protection. Thus, our goal is to balance these objectives. Given the complexity of solving this problem in a high-dimensional space, we employ experience replay and policy gradient optimization techniques, using Q-value function approximation to reduce computational complexity and derive the optimal offloading decision for each task. Although the underlying system model employs instantaneous metrics at each time slot, our DRL-based approach

Environment Interaction: At each time step t , the vehicle observes the current state s_t and selects an action a_t using the actor network with added exploration noise

$$a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t \quad (19)$$

where $\mathcal{N}_t$ is Gaussian noise that encourages exploration. The selected action determines the offloading decision and resource allocation.

State Transition and Reward: The environment transitions to the next state s_{t+1} based on the offloading action a_t . The reward r_t is computed using the hierarchical reward function described in the previous subsection.

Experience Replay: The transition (s_t, a_t, r_t, s_{t+1}) is stored in the replay buffer $\mathcal{D}$. A minibatch of N transitions is sampled uniformly from $\mathcal{D}$ for training.

Critic Network Update: The target Q-value is computed using the target networks

$$y_t = r_t + \gamma \min_{i=1,2} Q'_i(s_{t+1}|\mu'(s_{t+1}|\theta^{\mu'}))|_{\theta^{Q'_i}} \quad (20)$$

where γ is the discount factor. The critic networks are updated by minimizing the loss

$$L(\theta^{Q_i}) = \frac{1}{N} \sum_{j=1}^N (Q_i(s_j, a_j|\theta^{Q_i}) - y_j)^2. \quad (21)$$

Actor Network Update: The actor network is updated every d steps (delayed update) by maximizing the expected Q-value of the first critic network

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_{j=1}^N \nabla_a Q_1(s_j, a|\theta^{Q_1})|_{a=\mu(s_j|\theta^\mu)} \nabla_{\theta^\mu} \mu(s_j|\theta^\mu). \quad (22)$$

Target Network Update: The target networks are updated using soft updates

$$\theta' \leftarrow \tau\theta + (1 - \tau)\theta' \quad (23)$$

where $\tau \in (0, 1)$ is the soft update coefficient.

In summary, the specific steps of the algorithm are presented in Algorithm 1.

C. FL-Based Training Framework With DP

In the FedVTO framework, each vehicle acts as an autonomous learning agent that continuously collects local data (e.g., vehicle information, task information, and offloading decisions). Vehicles use local DRL agents to decide whether to execute computational tasks locally or offload them to nearby RSUs. The RSU layer does not contain DRL agents or perform aggregation; instead, RSUs solely provide computational resources for processing offloaded tasks with low latency, returning results to vehicles immediately after computation. Subsequently, vehicles use the received results along with their local data to update their DRL models locally. Vehicles then transmit these updated model parameters, rather than raw task data, directly to the cloud server layer, where global model aggregation is performed via FL. To further enhance privacy during aggregation, DP [33] is applied by adding calibrated noise to each vehicle's model update. After

Algorithm 1: TD3-Based Task Offloading Optimization

Input: Actor network μ_θ , critic networks Q_{ϕ_1}, Q_{ϕ_2} , target networks $\mu_{\theta'}, Q_{\phi'_1}, Q_{\phi'_2}$, replay buffer $\mathcal{D}$, exploration noise $\mathcal{N}(0, v)$, discount factor γ , soft update rate τ , update frequency d

Output: Optimized offloading policy μ_θ^*

for each episode do

for each vehicle $i = 1, 2, \dots, N$ do

Initialize state $s_i(t)$;

while not terminal do

Select action $a_i(t)$ with exploration:

$a_i(t) = \mu_\theta(s_i(t)) + \mathcal{N}(0, v)$;

Execute action $a_i(t)$, observe reward $r_i(t)$ and next state $s_i(t+1)$;

Store transition $\langle s_i(t), a_i(t), r_i(t), s_i(t+1) \rangle$ in replay buffer $\mathcal{D}$;

if $\text{size}(\mathcal{D}) > \text{batch size}$ then

Sample minibatch of transitions $\langle s, a, r, s' \rangle$ from $\mathcal{D}$;

Compute target value y_t using target networks according to Eq.(22);

Update critic networks by minimizing loss according to Eq.(23);

if $\text{step \% } d == 0$ then

Update actor network using policy gradient according to Eq.(24);

Update target networks with soft updates according to Eq.(25);

aggregation, the cloud server distributes the updated global DRL model back to all vehicles, thereby refining their offloading decisions and improving DRL agent performance. The detailed steps of FL with DP in the FedVTO framework are as follows:

Local Model Training: Each vehicle v_i independently trains on its task data $T_i(t)$ without sharing raw data with the central server. At each training iteration t , the server broadcasts the global model $P_g(t)$ to all participating vehicles K . Each vehicle then updates this model locally to produce a refined version $P_k(t)$ using its own data

$$P_k(t+1) = P_k(t) - \eta \nabla \mathcal{L}(P_k(t), T_i(t)) \quad (24)$$

where η is the learning rate, and $\mathcal{L}$ is the loss function reflecting the accuracy of offloading decisions based on local vehicle data. This process allows each vehicle to adapt the model to its own data distribution, improving its relevance and decision-making accuracy.

Model Aggregation: Once local training is complete, each vehicle sends its updated model $P_k(t)$ to the central server. Using the FedAvg algorithm, the server aggregates these updates to create a new global model $P_g(t+1)$

$$P_g(t+1) = \frac{1}{K} \sum_{k=1}^K P_k(t). \quad (25)$$

By only sharing the model parameters and not raw data, this aggregation reduces privacy risks and avoids the transmission of potentially sensitive information from individual vehicles. This aggregated global model reflects the diverse conditions and task types encountered across all vehicles in the network.

DP Mechanism: To further protect privacy during aggregation, DP adds random noise to each vehicle's model parameters before transmission. Let $\Delta P_k(t)$ represent the model update from vehicle v_i ; DP ensures ϵ -DP by adding noise $\mathcal{N}(0, \sigma^2)$

$$\tilde{P}_k(t) = P_k(t) + \mathcal{N}(0, \sigma^2). \quad (26)$$

Here, σ is the noise scale, calibrated based on the sensitivity of $\Delta P_k(t)$ and the desired privacy level ϵ . This added noise helps obscure individual model updates, making it difficult to infer specific details about any vehicle's data from the aggregated global model.

Global Model Update and Deployment: The central server iteratively updates the global model $P_g(t+1)$ by repeating the local training and aggregation process over multiple rounds, distributing the improved model to all vehicles in each cycle.

The overall procedure of the FL-based task offloading framework—where edge servers execute tasks and the cloud server aggregates models—is detailed in Algorithm 2.

V. EXPERIMENTS

In the simulation, 25 vehicles are deployed, each continuously generating tasks across four distinct types and sizes to represent varying levels of computational complexity. In addition, 5 RSUs are strategically distributed throughout the region to manage task offloading from nearby vehicles. The detailed simulation parameters, including the number of vehicles, RSU placement, wireless channel models, CPU frequencies, and DP noise levels, are presented in Table II.

Each vehicle in the system is represented by an intelligent agent, which comprises four neural networks: 1) an Actor network; 2) two Critic networks (Critic 1 and Critic 2); 3) a Target Actor network; and 4) two Target Critic networks (Target Critic 1 and Target Critic 2). The Actor network consists of five fully connected layers. The input layer has a number of neurons equal to the dimensionality of the state vector, while the subsequent layers contain 256, 512, and 128 neurons, respectively, each utilizing the ReLU activation function. The output layer uses a Tanh activation function to generate action vectors. The Critic networks take input with dimensionality equal to the sum of the state and action vector dimensions, and their architecture mirrors that of the Actor network, except that the output layer produces a scalar value representing the state-action value (Q-value). The dual Critic networks help reduce Q-value estimation bias, while the Target Critic networks have identical architectures to the Critic networks. Optimization is performed using the Adam optimizer, with a learning rate of 0.0001 for the Actor network and 0.001 for the Critic networks [39]. The training process spans 2000 episodes, each comprising 500 time steps. A batch size of 64 is used, and the global experience replay buffer is set to 10^4 .

Algorithm 2: FedVTO Algorithm

Input: Task set $\mathcal{T}$, device states $\mathcal{S}$, privacy parameter ϵ
Output: Task average response time, system total energy consumption, and total privacy entropy
Initialize: Global model parameters $P_g(0)$ at cloud server at time $t = 0$
Cloud Server: Broadcast $P_g(0)$ to all edge servers
for each global iteration $t = 1, 2, \dots, T$ **do**
 Edge Server::
 for each edge server $e \in \mathcal{E}$ **in parallel do**
 Distribute $P_g(t)$ to connected devices;
 for each device $k \in \mathcal{K}_e$ **connected to edge server** e **do**
 Receive model $P_g(t)$ from the edge server;
 Set local model $P_k(t) = P_g(t)$;
 for each task $T_k(t)$ **generated by device** k **do**
 Select a computing node for $T_k(t)$ using offloading decision policy $\pi(a_k(t))$;
 Execute the task on the selected node;
 Train the local model $P_k(t)$ using task data;
 Add differential privacy noise:
 $\tilde{P}_k(t+1) = P_k(t+1) + \mathcal{N}(0, \sigma^2)$;
 Upload the updated model $\tilde{P}_k(t+1)$ to the edge server;
 Cloud Server::
 Receive model updates $\{\tilde{P}_k(t+1)\}_{k=1}^K$ from all edge servers;
 Aggregate global model using federated averaging::
 $P_g(t+1) = \frac{1}{K} \sum_{k=1}^K \tilde{P}_k(t+1)$;
 Broadcast the updated global model $P_g(t+1)$ to all edge servers;

TABLE II
SIMULATION PARAMETERS

Parameter	Value
B_i^{up}	[5,10] MHz
B_i^{down}	[8,15] MHz
P_i^{up}	[2,5] W
P_j^{down}	[5,10] W
$h_{i,j}$	[1,10]
f_i	[1,3] GHz
f_{r_j}	[5,10] GHz
$\delta_{\text{size}}^{\text{up}}$	[1,20] MB
$\delta_{\text{size}}^{\text{res}}$	[0.1,1] MB
σ^2	10^{-3} dBm
ϵ	10^{-2}

To evaluate the performance of the proposed DRL algorithms, we compare them with two additional algorithms: 1) DDPG[40] and 2) deep Q-network (DQN)[41]. DDPG extends the DQN framework to handle continuous action spaces by employing an actor-critic architecture, which allows

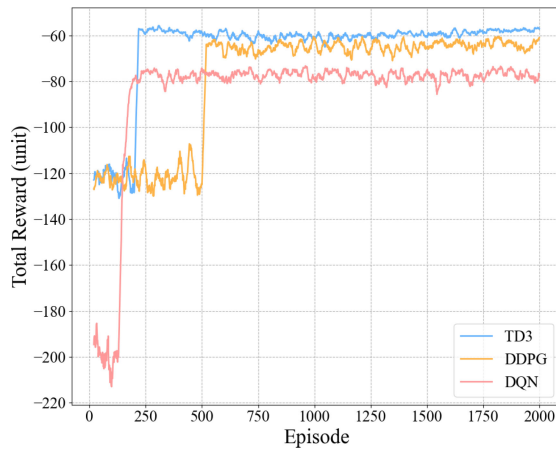


Fig. 3. Convergence of Total Rewards.

for effective policy learning in high-dimensional environments. However, DDPG is known to be sensitive to hyperparameter configurations and may experience instability during training. In contrast, DQN excels in environments with discrete action spaces, leveraging value-based methods to estimate the optimal action-value function and demonstrating strong performance in various discrete tasks. Nevertheless, DQN faces limitations when handling high-dimensional or continuous action spaces. In our experiments, we use TD3 as the DRL algorithm within the FedVTO framework, where TD3 is integrated with FL and an entropy-based weighting method to dynamically balance latency, energy consumption, and privacy. Therefore, in this article, the term TD3 refers to the full FedVTO system, which combines these components for effective task offloading and resource management. By comparing FedVTO with DDPG and DQN, we aim to evaluate its performance and stability across varying levels of action space complexity, thereby validating its effectiveness relative to these established algorithms.

A. Algorithm Validity Evaluation

To validate the effectiveness and robustness of the proposed method, we first illustrate the convergence behavior of key performance metrics, including total reward, overall latency, energy consumption, and privacy preservation, across the three representative benchmark algorithms. Then, we conduct a detailed analysis of how variations in the number of vehicles, edge servers, and available bandwidth affect the system's average latency, average energy consumption, and average privacy level. Finally, we investigate the impact of the entropy-based dynamic weighting strategy on the evolution of weight coefficients and assess how these temporal changes influence overall system performance.

1) *Convergence*: As shown in Fig. 3, the total reward of the TD3 algorithm stabilizes quickly in the early stages of training, reaching approximately -60 after about 250 iterations. In contrast, the DDPG algorithm converges more gradually, stabilizing near -65 after roughly 500 iterations. The DQN algorithm converges at a rate similar to TD3, but stabilizes

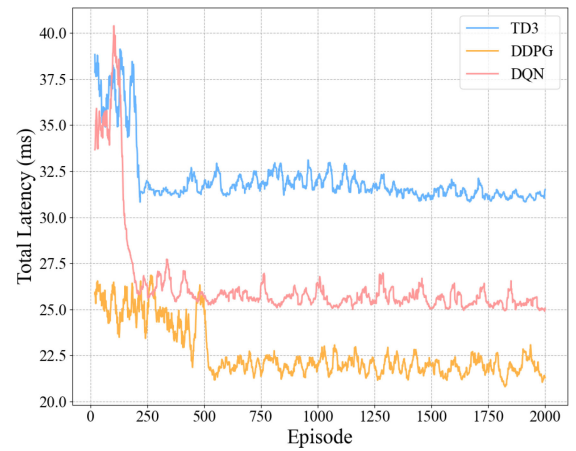


Fig. 4. Convergence of Total Latency.

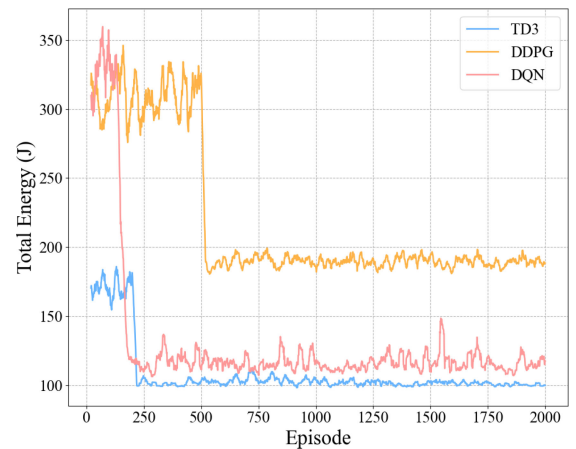


Fig. 5. Convergence of Total Energy.

around -76 after about 250 iterations. Overall, TD3 achieves the highest reward performance compared to DDPG and DQN.

In Fig. 4, the convergence behavior of total latency follows a similar pattern to that of the total reward. Early in training, TD3's latency fluctuates at higher values but stabilizes around 37 ms after approximately 250 steps. The DDPG algorithm shows similar behavior, though with slightly lower latency, stabilizing around 32 ms after 500 steps. The DQN algorithm starts with a slightly lower latency, but takes longer to stabilize, reaching about 30 ms after around 1500 steps. Overall, DDPG achieves the best latency optimization, followed by DQN, while TD3 ranks third in this metric.

As seen in Fig. 5, the convergence rate for energy consumption follows a similar trend to latency. The TD3 algorithm performs best, eventually stabilizing at around 100 J. This demonstrates TD3's superior efficiency in managing energy consumption, achieved through its ability to effectively balance exploration and exploitation. The use of delayed updates and target networks reduces energy-intensive actions, leading to optimal energy usage. The DQN algorithm ranks second with a convergence value of approximately 115 J, reflecting higher energy consumption than TD3 due to its less refined policy updates. In contrast, DDPG exhibits the highest energy consumption, converging at around 190 J. This higher

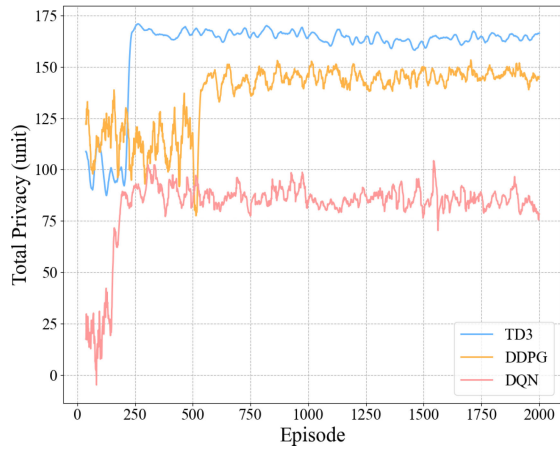


Fig. 6. Convergence of Total Privacy.

consumption results from DDPG's suboptimal actions during exploration, which lead to unnecessary energy expenditure.

As shown in Fig. 6, TD3 achieves the highest privacy level, converging near 260. In comparison, DDPG stabilizes at around 190, and DQN records the lowest privacy level, around 135. This is because TD3 prioritizes privacy-preserving actions even at the expense of slight latency tradeoffs. Although TD3 does not lead in latency, its strong privacy protection and energy efficiency make it superior in overall system objectives. DDPG focuses more on latency reduction, sometimes at the expense of privacy and energy, while DQN favors energy efficiency but struggles with both latency and privacy. Overall, TD3 offers the most balanced performance among the three algorithms, effectively addressing the tradeoffs between latency, energy, and privacy.

2) *Objectives*: Fig. 7 examines the impact of increasing the number of vehicles (10–50) on the performance of the TD3, DDPG, and DQN algorithms. As the number of vehicles increases, all algorithms experience a rise in average latency, average energy consumption, and average privacy leakage due to higher task complexity and resource contention. While TD3 performs poorly in terms of average latency, it achieves the best results in both energy consumption and privacy protection. DDPG excels in latency optimization, but falls short of TD3 in energy consumption and privacy protection. On the other hand, DQN exhibits the highest latency and privacy leakage rates, making it less suitable for high-demand scenarios. These results suggest that TD3 demonstrates better scalability and efficiency in managing complex vehicular environments, making it well-suited for dynamic vehicular network settings.

Fig. 8 illustrates the effect of increasing the number of edge servers (5–25) on average latency, energy consumption, and privacy leakage. With fewer edge servers, all algorithms face higher latency, increased energy consumption, and greater privacy leakage, primarily due to limited computational resources and potential network congestion. As the number of edge servers increases, these metrics show substantial improvements. TD3 outperforms the others, achieving the best results in both privacy protection and energy efficiency. DDPG shows balanced improvements across all three metrics,

performing well in scenarios with moderate to high numbers of edge servers. However, DQN struggles to achieve comparable performance, particularly in terms of latency and privacy protection, lagging behind TD3 and DDPG. This highlights that while DQN is efficient in some respects, it is less adaptable in managing dynamic conditions, especially when privacy and latency are prioritized. By contrast, TD3's focus on privacy preservation and energy efficiency allows it to consistently maintain high performance, even in more complex network environments with more edge servers.

Fig. 9 illustrates how varying total bandwidth (10–50 MHz) affects the average latency, energy consumption, and privacy leakage for the TD3, DDPG, and DQN algorithms. As the bandwidth increases from 10 to 40 MHz, all three algorithms show a significant reduction in both latency and energy consumption, reflecting the alleviation of communication bottlenecks and more efficient resource utilization. Within this range, privacy protection also improves notably, highlighting the role of increased bandwidth in enhancing data security. However, beyond 40 MHz, the improvements in all metrics become minimal, indicating that further increases in bandwidth provide only limited additional benefits. Throughout the entire bandwidth range, TD3 consistently achieves the lowest latency, minimal energy consumption, and the highest level of privacy protection. While DDPG produces competitive results, it slightly lags behind TD3, particularly in terms of energy consumption and privacy. Although DQN improves as bandwidth increases, it consistently shows higher latency and energy consumption compared to the other algorithms.

During the initial training phase, all three algorithms start with identical weight assignments and adjust these weights dynamically every 100 episodes using an entropy-based method to balance latency, energy consumption, and privacy preservation. Fig. 10 shows the temporal evolution of the weighting parameters α , β , and γ , where each algorithm ensures that $\alpha + \beta + \gamma = 1$ throughout the training process. For TD3, the increasing area for the privacy weight (γ) indicates a growing emphasis on privacy protection, while the weights for latency (α) and energy (β) gradually decrease. In contrast, DDPG shows a consistent dominance of the latency weight (α), reflecting its focus on minimizing latency with limited adjustment to the other objectives. Meanwhile, DQN exhibits a steady rise in the energy weight (β), highlighting its primary focus on energy optimization. Overall, the stacked area chart illustrates distinct algorithmic priorities: DDPG prioritizes latency, TD3 emphasizes privacy, and DQN concentrates on energy efficiency.

Fig. 11 presents the effects of different weight configurations on total latency, total energy consumption, and total privacy. As shown in the figure, varying the weight parameters leads to noticeable differences among the three algorithms in terms of optimization objectives. When the weight disproportionately favors latency, DDPG achieves the lowest system latency but compromises on energy consumption and privacy protection. In contrast, when β is relatively large, DQN demonstrates an advantage in energy optimization—exhibiting lower overall energy consumption—albeit with lower privacy protection compared to the other two algorithms.

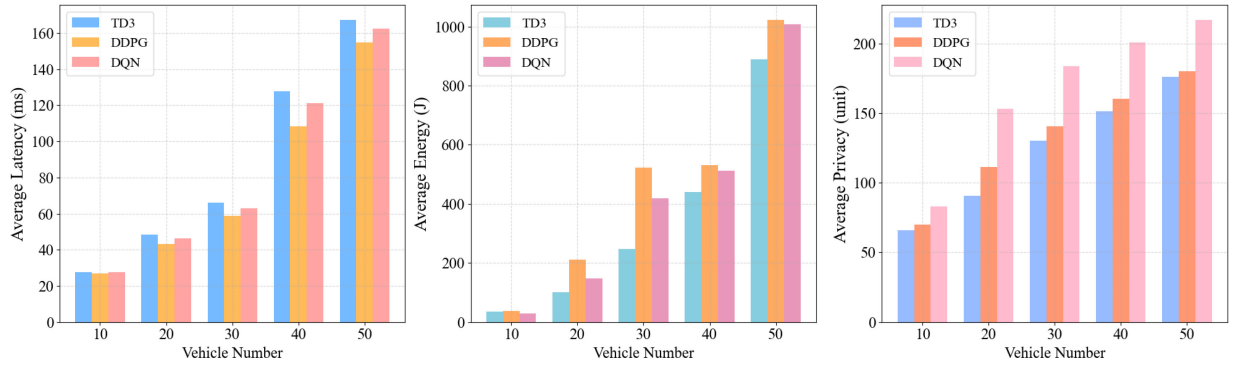


Fig. 7. Impact of Vehicle Number on Average Latency, Average Energy, and Average Privacy.

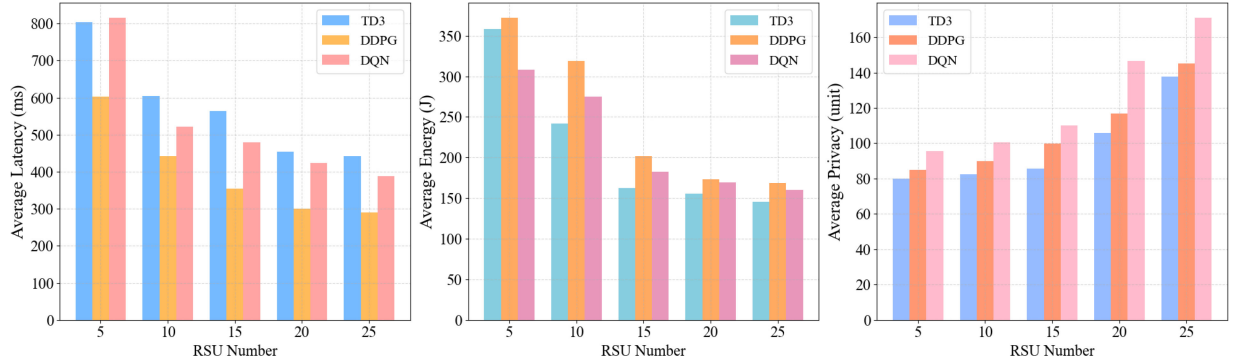


Fig. 8. Impact of RSU Number on Average Latency, Average Energy, and Average Privacy.

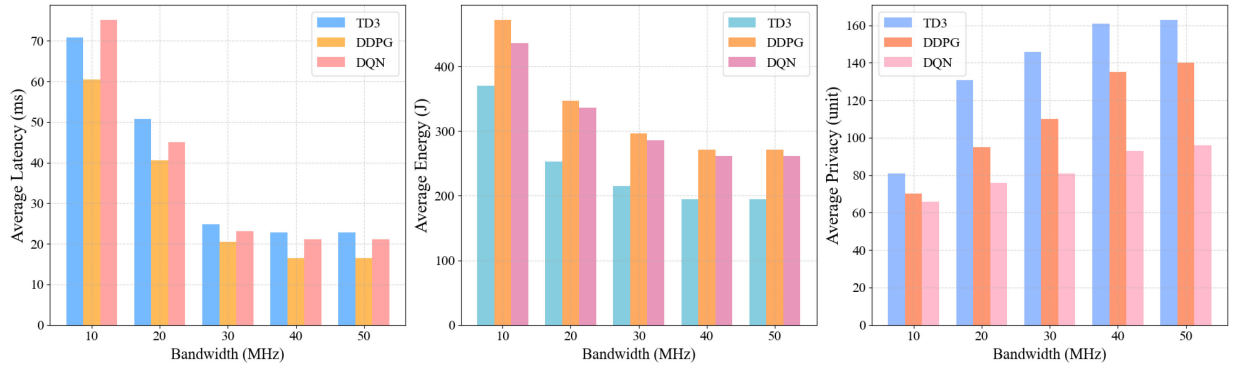


Fig. 9. Impact of total bandwidth on average latency, average energy consumption, and average privacy.

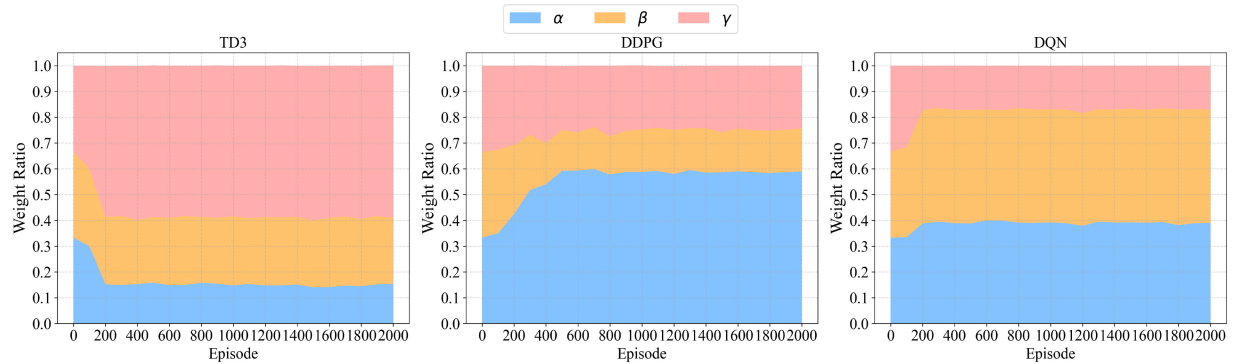


Fig. 10. Changes in weight parameters α , β , and γ .

Meanwhile, when privacy is given a higher weight, TD3 clearly outperforms the others in terms of privacy protection and maintains the lowest energy consumption, albeit with a slight increase in latency. Therefore, these results indicate

that different weight configurations lead to distinct tradeoffs among latency, energy consumption, and privacy. Specifically, DDPG is more inclined toward latency optimization, TD3 prioritizes privacy protection while also ensuring relatively low

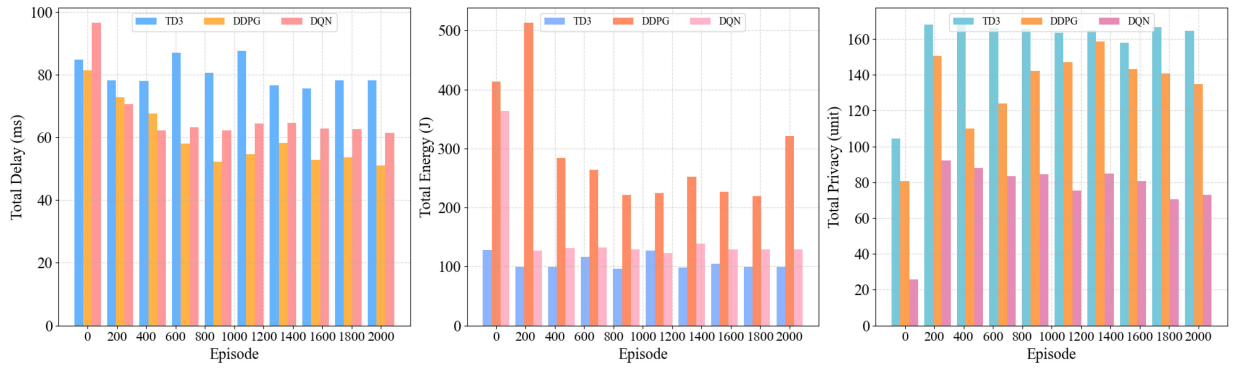


Fig. 11. Impact of different weight configurations on total latency, total energy consumption, and total privacy.

energy consumption, and DQN balances energy and latency but performs less competitively in preserving privacy.

VI. CONCLUSION

In this article, we present FedVTO, a federated task offloading framework designed for VEC networks. The framework aims to reduce latency, energy consumption, and privacy risks by jointly optimizing task offloading decisions and resource allocation. By integrating FL and DRL, we introduce a privacy entropy metric to evaluate and enhance privacy protection during task offloading. We model the task offloading problem as an optimization problem and solve it using the TD3 algorithm. Experimental results demonstrate that FedVTO significantly outperforms traditional methods in latency, energy efficiency, and privacy protection. These findings highlight that FedVTO provides an efficient, privacy-aware, and robust task offloading solution for dynamic vehicular environments. Future work will focus on enhancing scalability and incorporating advanced privacy protection techniques to further improve system performance.

REFERENCES

- [1] M. K. Hasan et al., "Federated learning for computational offloading and resource management of vehicular edge computing in 6G-V2X network," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 3827–3847, Feb. 2024.
- [2] T. J. Chua, W. Yu, and J. Zhao, "Play to earn in augmented reality with mobile edge computing over wireless networks: A deep reinforcement learning approach," *IEEE Trans. Wireless Commun.*, vol. 24, no. 1, pp. 68–83, Jan. 2025.
- [3] G. Yan, K. Liu, C. Liu, and J. Zhang, "Edge intelligence for Internet of Vehicles: A survey," *IEEE Trans. Consum. Electron.*, vol. 70, no. 2, pp. 4858–4877, May 2024.
- [4] M. Y. Akhlaqi and Z. B. Mohd Hanapi, "Task offloading paradigm in mobile edge computing-current issues, adopted approaches, and future directions," *J. New. Comput. Appl.*, vol. 212, Mar. 2023, Art. no. 103568. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1084804522002090>
- [5] Z. Liu et al., "PPRU: A privacy-preserving reputation updating scheme for cloud-assisted vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 74, no. 2, pp. 1877–1892, Feb. 2025.
- [6] M. Chen, A. Liu, N. N. Xiong, Y. Ren, and H. H. Song, "ANNS: An intelligent advanced non-convex non-smooth scheme for IRS-aided next generation mobile communication networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 9, pp. 8959–8973, Sep. 2025.
- [7] Y. Sun et al., "PPDR: A privacy-preserving dual reputation management scheme in vehicle platoon," *IEEE Trans. Dependable Secure Comput.*, early access, Jul. 15, 2025, doi: [10.1109/TDSC.2025.3588717](https://doi.org/10.1109/TDSC.2025.3588717).
- [8] J. Yang et al., "Service-aware computation offloading for parallel tasks in VEC networks," *IEEE Internet Things J.*, vol. 12, no. 3, pp. 2979–2993, Feb. 2025.
- [9] G. Sun, Z. Wang, H. Su, H. Yu, B. Lei, and M. Guizani, "Profit maximization of independent task offloading in MEC-enabled 5G Internet of Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 11, pp. 16449–16461, Nov. 2024.
- [10] M. Chen, A. Liu, N. N. Xiong, H. Song, and V. C. M. Leung, "SGPL: An intelligent game-based secure collaborative communication scheme for metaverse over 5G and beyond networks," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 3, pp. 767–782, Mar. 2024.
- [11] L. Yin, J. Luo, C. Qiu, C. Wang, and Y. Qiao, "Joint task offloading and resources allocation for hybrid vehicle edge computing systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 8, pp. 10355–10368, Aug. 2024.
- [12] G. G. Devarajan, S. Thangam, M. J. Alenazi, U. Kumaran, G. Chandran, and A. K. Bashir, "Federated learning and blockchain-enabled framework for traffic rerouting and task offloading in the Internet of Vehicles (IoV)," *IEEE Trans. Consum. Electron.*, vol. 71, no. 2, pp. 3817–3825, May 2025.
- [13] Y. Liu et al., "Vertical federated learning: Concepts, advances, and challenges," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 7, pp. 3615–3634, Jul. 2024.
- [14] T. Jiang, Z. Chen, Z. Zhao, M. Feng, and J. Zhou, "Deep reinforcement learning based task offloading and resource allocation in mobile edge computing network with heterogeneous tasks," *IEEE Internet Things J.*, vol. 12, no. 8, pp. 10899–10906, Apr. 2025.
- [15] R. Zhang, C. Pan, Y. Wang, Y. Yao, and X. Li, "Federated deep reinforcement learning for multimedia task offloading and resource allocation in MEC networks," *IEICE Trans. Commun.*, vol. E107-B, no. 6, pp. 446–457, Jun. 2024.
- [16] Y. Long, Z. Wang, S. Lan, R. Zhang, and K. Xu, "Energy-latency tradeoff for task offloading and resource allocation in vehicular edge computing," *Comput. Netw.*, vol. 258, Feb. 2025, Art. no. 111026.
- [17] F. Liu, J. Huang, and X. Wang, "Joint task offloading and resource allocation for device-edge-cloud collaboration with subtask dependencies," *IEEE Trans. Cloud Comput.*, vol. 11, no. 3, pp. 3027–3039, Jul.–Sep. 2023.
- [18] F. Chai, Q. Zhang, H. Yao, X. Xin, R. Gao, and M. Guizani, "Joint multi-task offloading and resource allocation for mobile edge computing systems in satellite IoT," *IEEE Trans. Veh. Technol.*, vol. 72, no. 6, pp. 7783–7795, Jun. 2023.
- [19] H. Yuan et al., "Cost-efficient task offloading in mobile edge computing with layered unmanned aerial vehicles," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 30496–30509, Oct. 2024.
- [20] G. Ma et al., "Joint partial offloading and resource allocation for vehicular federated learning tasks," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 8, pp. 8444–8459, Aug. 2024.
- [21] S. Shen, G. Shen, Z. Dai, K. Zhang, X. Kong, and J. Li, "Asynchronous federated deep-reinforcement-learning-based dependency task offloading for UAV-assisted vehicular networks," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 31561–31574, Oct. 2024.
- [22] H. Zhang, H. Zhao, R. Liu, A. Kaushik, X. Gao, and S. Xu, "Collaborative task offloading optimization for satellite mobile edge computing using multi-agent deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 73, no. 10, pp. 15483–15498, Oct. 2024.
- [23] M. Mohanapriya, V. Anusuya, K. Aravindhan, N. Krishnaveni, R. Santhosh, and D. Gowthami, "Optimizing task offloading in vehicular network (OTO): A game theory approach integrating hybrid edge and cloud computing," *J. Cybersec. Inf. Manage.*, vol. 15, no. 1, pp. 1–14, 2025.

- [24] Z. Abbas, S. Xu, and X. Zhang, "An efficient partial task offloading and resource allocation scheme for vehicular edge computing in a dynamic environment," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 2, pp. 2488–2502, Feb. 2025.
- [25] Y. Chen, J. Wu, S. Ye, W. Li, and Z. Xu, "Budget-constrained resource allocation and pricing in VEC: A MSMLMF Stackelberg game with contract incentive mechanism," *IEEE Internet Things J.*, vol. 12, no. 5, pp. 5050–5067, Mar. 2025. [Online]. Available: <https://api.semanticscholar.org/CorpusID:273975686>
- [26] Z. Chen, Y. Yang, J. Xu, Y. Chen, and J. Huang, "Task offloading and resource pricing based on game theory in UAV-assisted edge computing," *IEEE Trans. Services Comput.*, vol. 18, no. 1, pp. 440–452, Jan./Feb. 2025.
- [27] G. Wu et al., "Combining Lyapunov optimization with actor-critic networks for privacy-aware IIoT computation offloading," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 17437–17452, May 2024.
- [28] Z. Wang et al., "Location privacy-aware task offloading in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 3, pp. 2269–2283, Mar. 2024.
- [29] D.-G. Zhang, H.-Z. An, J. Zhang, T. Zhang, W.-M. Dong, and X.-R. Jiang, "Novel privacy awareness task offloading approach based on privacy entropy," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 3598–3608, Jun. 2024.
- [30] S. Liu et al., "Satisfaction-Maximized secure computation offloading in multi-eavesdropper MEC networks," *IEEE Trans. Wireless Commun.*, vol. 21, no. 6, pp. 4227–4241, Jun. 2022.
- [31] W. Shang, J. Ge, L. Ding, Z. Jiang, and H. Sui, "Acceleration offloading for differential privacy protection based on federated learning in edge intelligent controllers," *0Future Gener. Comput. Syst.*, vol. 163, Feb. 2025, Art. no. 107526. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X24004904>
- [32] S. Wang, J. Li, G. Wu, H. Chen, and S. Sun, "Joint optimization of task offloading and resource allocation based on differential privacy in vehicular edge computing," *IEEE Trans. Comput. Soc. Syst.*, vol. 9, no. 1, pp. 109–119, Feb. 2022.
- [33] P. Zhao, Z. Yang, and G. Zhang, "Personalized and differential privacy-aware video stream offloading in mobile edge computing," *IEEE Trans. Cloud Comput.*, vol. 12, no. 1, pp. 347–358, Jan.–Mar. 2024.
- [34] R. Zhang, R. Zhou, Y. Wang, H. Tan, and K. He, "Incentive mechanisms for online task offloading with privacy-preserving in UAV-assisted mobile edge computing," *IEEE/ACM Trans. Netw.*, vol. 32, no. 3, pp. 2646–2661, Jun. 2024.
- [35] W. Lan, K. Chen, Y. Li, J. Cao, and Y. Sahni, "Deep reinforcement learning for privacy-preserving task offloading in integrated satellite-terrestrial networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9678–9691, Oct. 2024.
- [36] F. Xia, Y. Chen, and J. Huang, "Privacy-preserving task offloading in mobile edge computing: A deep reinforcement learning approach," *Softw., Pract. Exp.*, vol. 54, no. 9, pp. 1774–1792, 2024.
- [37] Y. Shi et al., "Efficient federated learning with enhanced privacy via lottery ticket pruning in edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9946–9958, Oct. 2024.
- [38] C. Wu, L. Li, L. Zhang, C. Gao, X. Wu, and S. Xiao, "Efficient GAN-based federated optimization for vehicular task offloading with mobile edge computing in 6G network," *IEEE Internet Things J.*, vol. 12, no. 3, pp. 2736–2748, Feb. 2025.
- [39] G. Wu, X. Chen, Z. Gao, H. Zhang, S. Yu, and S. Shen, "Privacy-preserving offloading scheme in multi-access mobile edge computing based on MADRL," *J. Parallel Distrib. Comput.*, vol. 183, Jan. 2024, Art. no. 104775.
- [40] H. Gao, X. Wang, W. Wei, A. Al-Dulaimi, and Y. Xu, "Com-DDPG: Task offloading based on Multiagent reinforcement learning for information-communication-enhanced mobile edge computing in the Internet of Vehicles," *IEEE Trans. Veh. Technol.*, vol. 73, no. 1, pp. 348–361, Jan. 2024. [Online]. Available: <https://doi.org/10.1109/TVT.2023.3309321>
- [41] Z. Tong, J. Deng, J. Mei, Y. Zhang, and K. Li, "Multi-objective DAG task offloading in MEC environment based on federated DQN with automated hyperparameter optimization," *IEEE Trans. Services Comput.*, vol. 17, no. 6, pp. 3999–4012, Nov./Dec. 2024.